

WEEK 6

Apache Spark — Architecture & Data Processing

Complete Assignment Solutions with PySpark Code & Insights

SECTION 1: Spark Architecture

Q1: Roles of Driver, Cluster Manager, and Executor

Apache Spark's distributed architecture consists of three core components that coordinate to execute a Spark application:

Component	Location	Responsibility
Driver	Client/Master node	Orchestrates job: builds DAG, schedules tasks, collects results
Cluster Manager	Dedicated manager node	Allocates resources (CPU/RAM) across the cluster
Executor	Worker nodes	Executes tasks, caches data, reports status to Driver

Driver: The master process that runs the `main()` function. It converts user code into a DAG (Directed Acyclic Graph) of tasks, negotiates resources with the Cluster Manager, and schedules tasks on Executors. It also tracks the progress of all tasks.

Cluster Manager: Acts as the resource broker between the Driver and the cluster. It allocates CPU/memory resources to the Spark application. Common options include YARN, Mesos, Kubernetes, or Spark Standalone.

Executor: JVM processes running on worker nodes. Each executor runs tasks assigned by the Driver and stores data in memory (RDD cache) or disk for efficient reuse. Multiple executors can run in parallel across nodes.

Key Insight: If an Executor fails, the Driver re-schedules those tasks on another Executor using the Lineage Graph — no data loss occurs.

Q2: Lazy Evaluation and Performance Optimization

Lazy Evaluation means Spark does NOT execute transformations immediately. Instead, it builds a logical execution plan (DAG) and only triggers computation when an Action is called.

How it improves performance:

- Query Optimization: Spark's Catalyst Optimizer can rearrange operations (e.g., push filters before joins) to minimize data shuffled.
- Reduced Computation: Only the data paths needed for the final result are computed — unnecessary steps are pruned.
- Pipeline Fusion: Multiple transformations are merged into a single stage, avoiding redundant reads/writes.
- Fault Recovery: The full lineage is known upfront, enabling precise task re-execution after failures.

```
# No computation happens here — only a plan is recorded
df = spark.read.csv('data/source.csv', header=True)
df_filtered = df.filter(df['category'] == 'Electronics') # Lazy
df_selected = df_filtered.select('product_id', 'price') # Lazy

# Computation triggered only here (Action):
df_selected.show() # <-- This is when Spark actually runs
```

Q7: Lineage Graph (DAG) and Fault Tolerance

Spark maintains a Lineage Graph (DAG) that records every transformation applied to create a DataFrame. If a worker node fails:

- Spark does NOT need to restart the entire job.
- It traces back through the DAG to find the last known good parent RDD/DataFrame partition.
- It re-computes only the lost partitions from the source — not the full dataset.

```
# Lineage chain: source -> filter -> select -> groupBy
# If the groupBy partition is lost, Spark replays:
# source.csv -> filter(category='Electronics') -> select -> groupBy

df.explain() # Shows the full logical and physical plan
```

Insight: This is why Spark doesn't need HDFS replication for fault tolerance — the lineage IS the recovery mechanism.

Q13: Client Mode vs Cluster Mode

Execution mode determines where the Driver process runs:

Feature	Client Mode	Cluster Mode
Driver Location	Local machine (submitter)	On a cluster node
Network Dependency	Requires open connection to cluster	No local dependency once submitted
Best For	Interactive dev, Jupyter, testing	Production, scheduled batch jobs
Log Access	Logs stream to local console	Logs stored on cluster

Failure Impact	Job dies if local machine goes down	Job survives local machine shutdown
----------------	-------------------------------------	-------------------------------------

```
# Client Mode (Driver on local machine – good for dev/testing)
spark-submit --master yarn --deploy-mode client app.py

# Cluster Mode (Driver on cluster – good for production)
spark-submit --master yarn --deploy-mode cluster app.py
```

SECTION 2: File Formats & Schema Handling

Q3: Reading a CSV File with Header and InferSchema

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('Week6').getOrCreate()

df = spark.read \
    .option('header', 'true') \
    .option('inferSchema', 'true') \
    .csv('data/source.csv')

df.printSchema() # Verify inferred types
df.show(5)       # Preview first 5 rows
```

Note: inferSchema=True causes Spark to scan the file once to determine column types. For production, prefer providing an explicit schema to avoid this extra scan.

Q4: CSV vs Parquet — Row-Based vs Columnar Storage

Aspect	CSV (Row-Based)	Parquet (Columnar)
Storage Layout	Row by row	Column by column
File Size	Large (no built-in compression)	Small (column-level compression)
Read Performance	Reads all columns always	Reads only needed columns
Schema Support	None (everything is String)	Enforced, typed schema
Predicate Pushdown	Not supported	Fully supported
Human Readable	Yes (plain text)	No (binary format)
Best Use Case	Data exchange, exports, small files	Analytics, production pipelines

Why it matters for performance:

- **Columnar Reads:** If you only need columns A and B from a 100-column dataset, Parquet reads only those columns. CSV must read all 100 columns.
- **Compression:** Columnar format enables column-level compression (e.g., dictionary encoding), often achieving 10x smaller files.

- Predicate Pushdown: Parquet stores min/max statistics per row group, allowing Spark to skip entire blocks of data.

Q9: Predicate Pushdown in Parquet

Predicate Pushdown is an optimization where filter conditions are pushed down to the file storage layer BEFORE data is loaded into Spark memory.

How it works in Parquet:

- Each Parquet file stores Row Groups (typically 128MB each).
- Each Row Group has Footer Statistics (min/max/null count per column).
- When Spark sees filter(price > 1000), it checks the footer statistics.
- If a row group's max(price) < 1000, the entire row group is SKIPPED — never loaded into memory.

```
# Spark automatically applies predicate pushdown for Parquet
df = spark.read.parquet('path/to/data.parquet')
df_filtered = df.filter(df['price'] > 1000)

# Verify pushdown is happening:
df_filtered.explain() # Look for 'PushedFilters' in the plan

# Output will show:
# PushedFilters: [NotNull(price), GreaterThan(price,1000.0)]
```

Impact: With predicate pushdown, a query that touches 5% of data only reads 5% of disk — dramatic I/O reduction on terabyte datasets.

SECTION 3: DataFrame Transformations & Filtering

Q5: Select Columns with Filter Condition

```
# Select product_id and price where category is 'Electronics'
result = df.select('product_id', 'price') \
            .filter(df['category'] == 'Electronics')

# Alternative using SQL-style .where()
result = df.where(df.category == 'Electronics') \
            .select('product_id', 'price')

result.show()
```

Q6: Rename Column and Cast Data Type

```
from pyspark.sql.functions import col
from pyspark.sql.types import DoubleType

# Step 1: Rename column old_name -> new_name
# Step 2: Cast price from String to Double
```

```
df_revised = df \
    .withColumnRenamed('old_name', 'new_name') \
    .withColumn('price', col('price').cast(DoubleType()))

df_revised.printSchema() # Verify changes
df_revised.show(5)
```

Best Practice: Always cast numeric columns from String to their proper type (DoubleType, IntegerType) before aggregations. Arithmetic on StringType columns will fail or give wrong results.

Q8: Filter with AND Condition

```
# Filter: status == 'Completed' AND amount > 1000
df_result = df_orders.filter(
    (df_orders['status'] == 'Completed') &
    (df_orders['amount'] > 1000)
)

# Alternative using .where() with SQL string
df_result = df_orders.where("status = 'Completed' AND amount > 1000")

df_result.show()
```

Q10: Add a New Computed Column (18% Tax)

```
from pyspark.sql.functions import col, round as spark_round

# Add final_price = base_price * 1.18
df_with_tax = df.withColumn(
    'final_price',
    spark_round(col('base_price') * 1.18, 2) # 2 decimal places
)

df_with_tax.select('base_price', 'final_price').show(5)
```

Q11: Transformations vs Actions

Transformations are lazy operations that define a new DataFrame from an existing one. They build the DAG but do NOT execute immediately.

Type	Description	Examples
Transformation (Lazy)	Creates new DataFrame, no execution	filter(), select(), groupBy(), join(), withColumn()
Action (Eager)	Triggers DAG execution, returns result	show(), count(), collect(), write(), take()

```
# TRANSFORMATIONS (Lazy — no execution):
df2 = df.filter(df['age'] > 18) # Transformation
df3 = df2.select('name', 'age') # Transformation
df4 = df3.withColumn('senior', df3['age'] > 60) # Transformation
```

```
# ACTIONS (Eager – triggers all pending transformations):
df4.show()           # Action → executes the entire chain
count = df4.count()  # Action → executes and returns a Python int
data = df4.collect() # Action → brings all data to Driver (use carefully!)
```

Q14: Filter with OR Condition

```
# Filter: region == 'North' OR priority == 'High'
df_result = df.filter(
    (df['region'] == 'North') |
    (df['priority'] == 'High')
)

# Alternative using SQL expression string:
df_result = df.where("region = 'North' OR priority = 'High'")

df_result.show()
```

SECTION 4: Data Pipelines & Best Practices

Q12: Read Parquet → Filter Nulls → Write as CSV

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('NullFilter').getOrCreate()

# Step 1: Load Parquet file
df = spark.read.parquet('path/to/input')

# Step 2: Filter out rows where user_id is null
df_clean = df.filter(df['user_id'].isNotNull())

# Step 3: Save result as CSV
df_clean.write \
    .option('header', 'true') \
    .mode('overwrite') \
    .csv('path/to/output')

print(f'Rows written: {df_clean.count()}')
```

Note: Spark writes CSV/Parquet as a directory with part files (part-00000.csv etc.), not a single file. This is intentional — it enables parallel writing across executors.

Q15: Why .show(5) is Safer than .collect() on Large Datasets

On a multi-terabyte dataset, these two commands behave very differently:

Operation	.show(5)	.collect()
Data moved to Driver	5 rows only	ALL rows in dataset

Memory risk	None	OutOfMemoryError on TB data
Speed	Very fast	Slow (full data scan + network)
Use case	Exploration, debugging	Only on small, bounded DataFrames

```
# SAFE: .show(5) - fetches only 5 rows to Driver
df.show(5)           # Use for exploration
df.show(5, truncate=False) # Show full column values

# DANGEROUS on large datasets: .collect()
# data = df.collect() # DO NOT USE on TB-scale data

# If you must collect, always limit first:
data = df.limit(1000).collect() # Safe - bounded

# Other safe alternatives for exploration:
df.take(10)      # Returns 10 rows as a Python list
df.first()       # Returns first row only
df.describe().show() # Statistical summary (safe)
```

Rule of Thumb: Use `.show()` for development/debugging. Reserve `.collect()` only for small, bounded DataFrames where you are certain all data fits in the Driver's memory.

Summary: Key Concepts at a Glance

Q#	Topic	Key Takeaway
Q1	Architecture	Driver orchestrates, Cluster Manager allocates, Executors compute
Q2	Lazy Evaluation	Transformations build a plan; Actions trigger optimized execution
Q3	Read CSV	<code>spark.read.option('header','true').option('inferSchema','true').csv(path)</code>
Q4	CSV vs Parquet	Parquet is columnar – faster reads, smaller size, predicate pushdown
Q5	Select + Filter	<code>df.select('col1','col2').filter(df['col'] == 'value')</code>
Q6	Rename + Cast	<code>withColumnRenamed() + withColumn(col.cast(DoubleType()))</code>
Q7	Fault Tolerance	DAG Lineage re-computes lost partitions without full job restart
Q8	AND Filter	<code>filter((col1 == val1) & (col2 > val2))</code>
Q9	Predicate Pushdown	Parquet row-group statistics skip irrelevant data blocks at I/O layer
Q10	New Column	<code>withColumn('final_price', col('base_price') * 1.18)</code>

Q1 1	Transform vs Action	Transformations are lazy (filter/select); Actions trigger execution (show/count)
Q1 2	Pipeline	<code>read.parquet() → filter(isNotNull()) → write.csv()</code>
Q1 3	Deploy Mode	Client: Driver local (dev); Cluster: Driver on cluster (production)
Q1 4	OR Filter	<code>filter((col1 == val1) (col2 == val2))</code>
Q1 5	show vs collect	<code>.show(5)</code> is safe; <code>.collect()</code> can crash Driver with large data